
Introduction to Database
DS350

Project

Deadline: Monday 09/08/2026 @ 23:59

[Total Mark is 20]

Student Details:

CRN:50300

Name: Sara Khalid Alnwaihel

ID: 240019342

Name: Lena Turki ALQURAINI

ID: 240036676

Name: Jana Mousa Haqawi

ID: 240044047

Name: Mayar Abdulrahman Alshehri

ID: 230027261

Instructions:

- You must submit two separate copies (**one Word file and one PDF file**) using the Assignment Template on Blackboard via the allocated folder. These files **must not be in compressed format**.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Zero mark will be given if you try to bypass the SafeAssign (e.g. misspell words, remove spaces between words, hide characters, use different character sets, **convert text into image** or languages other than English or any kind of manipulation).
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template, failing which will result in zero mark.
- You **MUST** show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- Late submission will result in ZERO mark.
- The work should be your own, copying from students or other resources will result in ZERO mark.
- Use **Times New Roman** font for all your answers.

Project Instructions

- You can work on this project as a group (minimum 3 and maximum 4 students). Each group member must submit the project individually with all group member names mentioned on the cover page.
- This project **worth 20 marks** and will be distributed as in the following:
 - Database Design. (6 marks)
 - Database Implementation. (6 marks)
 - SQL Operations. (6 marks)
 - Project Reflection. (2 marks)
- Each leader in the group must submit one report about their chosen Project via the Blackboard (**Email submission will not be accepted and will be awarded ZERO marks**) containing the following:
 - A. Task 1 : database design
 - B. Task 2: database implementation
 - C. All requested queries/results
 - D. Screenshots from MySQL (or any other software you use) of **all the tables** after population and **query results**.
 - E. Reflection (300–500 words) describing the team's experience, challenges, design decisions, and AI usage (if applicable).
- Submit one **SQL file** containing the complete database implementation, including database creation, tables, data insertion, required queries, VIEW, and TRIGGER.
- Submit a **GitHub** (or GitLab) repository containing the complete project with regular commits and meaningful commit messages.
- Submit a **progress report** describing the completed work, key decisions, challenges, future plan, and contribution of each group member.
- Include **links** to the live report document, GitHub/GitLab repository, and Google Colab notebook (if applicable) in the **Project Artifacts** section of the final report.
- You are advised to make your work clear and well presented; marks may be reduced for poor presentation. This includes filling in your information on the cover page.
- You MUST show all your work, and text **must not** be converted into an image unless specified otherwise by the question.
 - **Late submission will result in ZERO marks being awarded.**
 - The work should be your own. Copying **from students or other resources, including AI software, will result in ZERO marks.**

*Learning**Outcome(s):**CLO1, CLO2, CLO3,**CLO4, CLO5***Project Description**

Design and Implementation of a Smart Clinic Database System

A private clinic currently manages its patient information manually, making it difficult to organize appointments, treatments, and payments. The clinic has decided to develop a database system to improve data organization and retrieval.

Your team is required to design, implement, and test a complete relational database using the concepts learned throughout this course. The project should demonstrate your understanding of database design, ER/EER modeling, relational mapping, SQL implementation, and advanced SQL queries. The project must be completed using MySQL.

Task 1 – Database Design (6 Marks)

Design a complete database for the clinic that includes at least six entities such as Patients, Doctors, Appointments, Treatments, Medicines, Payments, or any other appropriate entities.

A complete ER/EER diagram.

Primary and foreign keys.

Appropriate relationships with cardinality.

At least one specialization/generalization (EER feature).

Any assumptions made during the design process.

Task 2 – Database Implementation (6 Marks)

Complete the following tasks using MySQL. Include screenshots of both your SQL code and the corresponding output (results) for each task:

Create the database and all required tables.

Use appropriate data types and constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, etc.).

Insert at least 5 records into each main table.

Task 3 – SQL Operations (6 Marks)

Complete the following tasks using MySQL. For each task, include screenshots of both your SQL code and the corresponding output (results), along with a brief description (one to two sentences) explaining the purpose of the query and the information it is intended to retrieve.

Write SELECT statements.

Write JOIN queries.

Write nested queries.

Use aggregate functions with GROUP BY.

Write UPDATE and DELETE statements.

Create one VIEW.

Create one TRIGGER.

Task 4 – Reflection (2 Marks)

Write a **300–500-word reflection** on your experience while completing this project by answering the following questions. Your responses should be clear, concise, and supported with specific examples from your own work.

Describe one specific challenge or obstacle you encountered during this project and explain in detail how you identified and resolved it.

Explain why you chose your specific approach/method/model over at least one alternative you considered, and what tradeoffs informed that decision.

If you used any AI tools during this project (e.g., ChatGPT, Claude, Copilot), disclose which tool(s), for what specific purpose (e.g., debugging, grammar checking, brainstorming), and how you verified or modified the output.

What would you do differently if you were to redo this project with more time or resources?

Deliverables

Project Report (PDF) – System description, ER/EER diagram, relational schema, SQL code, screenshots, and reflection.

SQL Script (.sql) – Complete SQL implementation of the project.

GitHub Repository – Repository link with the complete project and commit history.

Mid-Project Progress Report – Summary of progress, challenges, decisions, and team contributions.

Project Artifacts

Live report document (with edit history): [\[link\]](#)

GitHub/GitLab repository (with commit history): [\[link\]](#)

Google Colab notebook (if applicable): [\[link\]](#)

Smart Clinic Database System

Introduction

A private clinic currently manages patient information manually, which makes it difficult to organize appointments, treatments, medicines, and payment records efficiently. As the amount of information grows, manual record management can lead to data duplication, errors, and delays when retrieving important information. To improve the management of clinic data, a database system is needed to store and organize information in a structured and reliable way.

This project focuses on the design and implementation of a Smart Clinic Database System. The system is intended to support the clinic's daily operations by managing patient records, doctor information, appointments, treatments, medicines, and payments. Using a relational database allows information to be stored consistently while making it easier to access and manage when needed.

Throughout this project, database concepts learned in the course are applied, including ER and EER modeling, relational schema design, SQL implementation, and advanced SQL operations. The database will be implemented using MySQL and will include tables, relationships, constraints, queries, a view, and a trigger. The completed system aims to provide an organized and efficient solution for managing clinic information while demonstrating the practical application of database design principles.

Task 1: Database Design

Private clinics handle a large amount of information related to patients, doctors, appointments, treatments, medicines, and payments. Managing this information manually can make it difficult to maintain accurate records, organize data efficiently, and retrieve information when needed. To address these challenges, a database design is required to provide a structured and organized approach to data management.

For this task, we designed a Smart Clinic Database System using Entity Relationship (ER) and Enhanced Entity Relationship (EER) modeling techniques. The design identifies the main entities involved in the clinic's operations, defines their attributes, and establishes the relationships between them. Primary keys, foreign keys, and cardinality constraints were incorporated to ensure data integrity and accurately represent real-world clinic processes. In addition, specialization and generalization concepts were applied in the EER model to provide a more comprehensive representation of the system. The completed design serves as the foundation for the database implementation phase.

– Database Entities

The ER model of the Smart Clinic Database System consists of the following entities:

- Patients
- Doctors
- Appointments
- Treatments
- Medicines
- Payments

Each entity contains attributes relevant to its role within the clinic and is uniquely identified by a primary key.

To enhance the database design, we introduced a superclass named **PERSON** in the EER model to represent the common attributes shared by Patients and Doctors. The specialization uses an overlapping constraint, allowing an individual to be both a Patient and a Doctor simultaneously, and a partial completeness constraint, allowing a Person to belong to neither subclass. This approach reduces redundancy and provides a more realistic representation of the clinic environment.

– ER Diagram

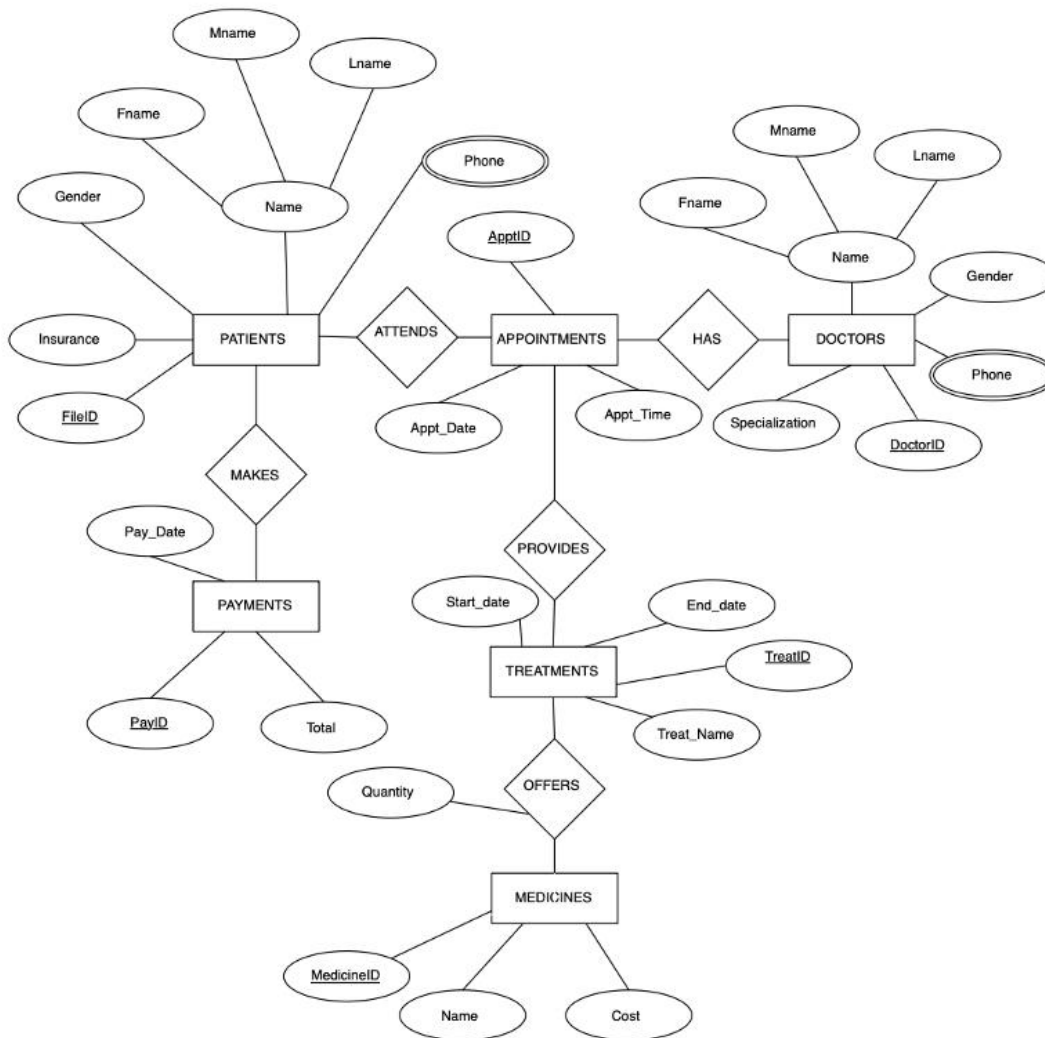


Figure 1: ER Diagram of the Smart Clinic Database System.

The ER diagram illustrates the main entities, attributes, primary keys, relationships, and cardinality constraints of the Smart Clinic Database System. It provides a conceptual representation of the data required to manage patients, doctors, appointments, treatments, medicines, and payments within the clinic.

– EER Diagram

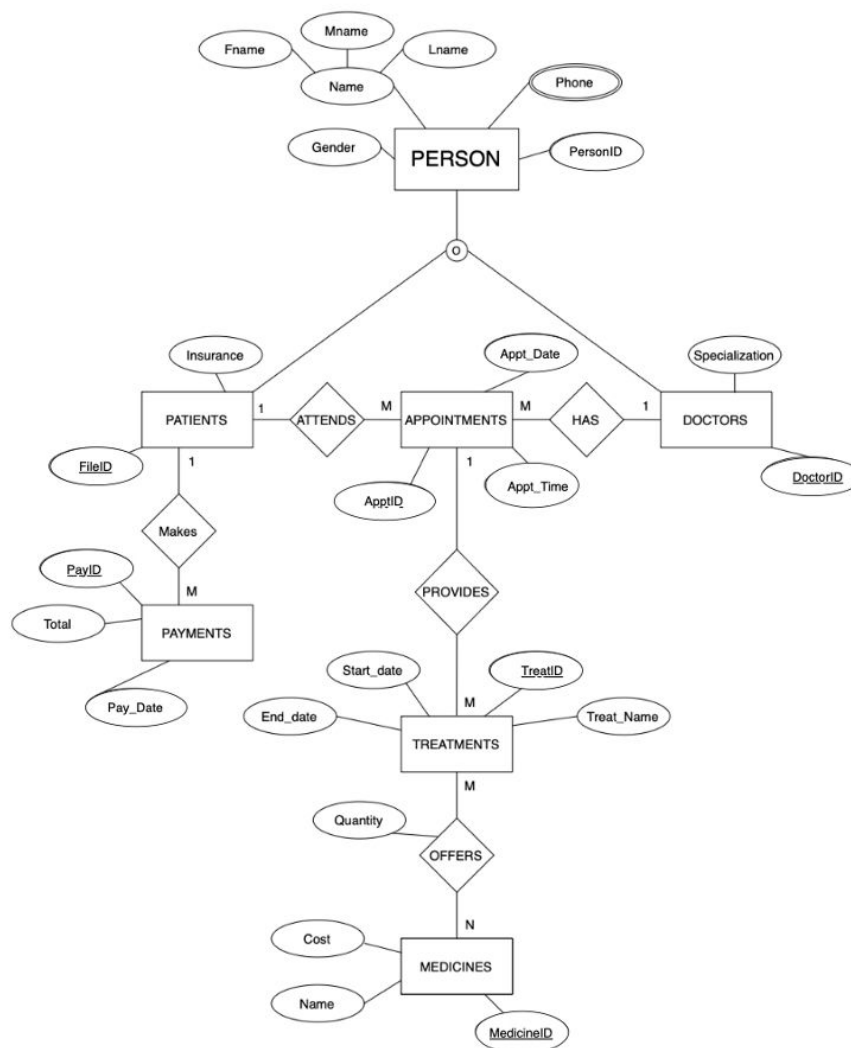


Figure 2: EER Diagram of the Smart Clinic Database

System

The EER diagram extends the ER model by applying the specialization/generalization concept. A Person superclass was introduced to represent common attributes shared by Patients and Doctors, while overlapping and partial specialization constraints were used to model the relationship between the superclass and its subclasses.

– Relationships and Cardinalities

We established several relationships between the entities to accurately represent the clinic's operations and ensure proper data organization.

- **Patient and Appointment**

A patient can have multiple appointments, while each appointment is associated with one patient. Therefore, a one-to-many (1:M) relationship exists between Patients and Appointments.

- **Doctor and Appointment**

A doctor can manage multiple appointments, while each appointment is assigned to one doctor. Therefore, a one-to-many (1:M) relationship exists between Doctors and Appointments.

- **Patient and Payment**

A patient can make multiple payments, while each payment belongs to one patient. Therefore, a one-to-many (1:M) relationship exists between Patients and Payments.

- **Appointment and Treatment**

An appointment may involve multiple treatments through the **Provide** relationship. Each treatment record is associated with a specific appointment.

- **Treatment and Medicine**

A treatment may include multiple medicines, and a medicine may be used in multiple treatments. Therefore, a many-to-many (M:N) relationship exists between Treatments and Medicines through the **Offers** relationship.

The quantity of medicine prescribed is stored as an attribute of the Offers relationship because the quantity may vary depending on the treatment in which the medicine is used.

– Design Assumptions

Several assumptions were made during the database design process.

We assumed that the Patient entity includes an attribute called **Insurance**, while the Doctor entity includes a **Specialization** attribute. In addition, **Phone** was modeled as a multivalued attribute for both Patient and Doctor entities. The primary key of each entity is identified by underlining the corresponding key attribute.

We also assumed that each Appointment may involve multiple Treatments, represented by the Provide relationship. Furthermore, each Treatment may include multiple Medicines, and the quantity of each medicine may vary depending on the prescribed treatment. Therefore, the quantity attribute was associated with the Offers relationship between Treatments and Medicines.

For the EER diagram, we applied the generalization/specialization concept. We assumed that every Person in the superclass may be a Patient, a Doctor, or neither of them. Consequently, a partial completeness constraint was used, represented by a single line connecting the superclass to the specialization.

Additionally, we assumed that an individual may simultaneously be both a Patient and a Doctor. Therefore, an overlapping constraint, represented by the symbol "o", was used instead of a disjoint constraint.

– Primary Keys

The following primary keys were used to uniquely identify records within the database:

<i>Entity</i>	<i>Primary Key</i>
<i>PERSON</i>	PersonID
<i>PATIENTS</i>	FileID
<i>DOCTORS</i>	DoctorID
<i>APPOINTMENTS</i>	ApptID
<i>TREATMENTS</i>	TreatID
<i>MEDICINES</i>	MedicineID
<i>PAYMENTS</i>	PayID

These primary keys ensure that each record can be uniquely identified and support the establishment of relationships among entities within the database.

– Summary

In this task, we designed a database for the Smart Clinic Database System using ER and EER modeling techniques. The design includes the required entities, attributes, relationships, primary keys, cardinality constraints, and specialization/generalization concepts needed to support clinic operations. The ER model provides the foundation of the database structure, while the EER model enhances the design through the use of a Person superclass and overlapping specialization for Patients and Doctors. This design will serve as the basis for the implementation and testing phases of the project using MySQL.

Task 2: Database Implementation

Following the completion of the database design presented in Task 1, the Smart Clinic Database System was implemented using MySQL. The implementation was based on the ER and EER models developed during the design phase. This stage involved creating the database, defining all required tables, establishing relationships through primary and foreign keys, applying appropriate constraints, and inserting sample records. The completed implementation transformed the conceptual database design into a functional relational database capable of supporting the clinic's daily operations while ensuring data consistency and integrity.

– Database Creation

The implementation process began by creating the **SmartClinicDB** database and selecting it as the active database. This database serves as the central repository for all tables and data required by the Smart Clinic Database System.

SQL Code:

```
CREATE DATABASE SmartClinicDB;
USE SmartClinicDB;
```

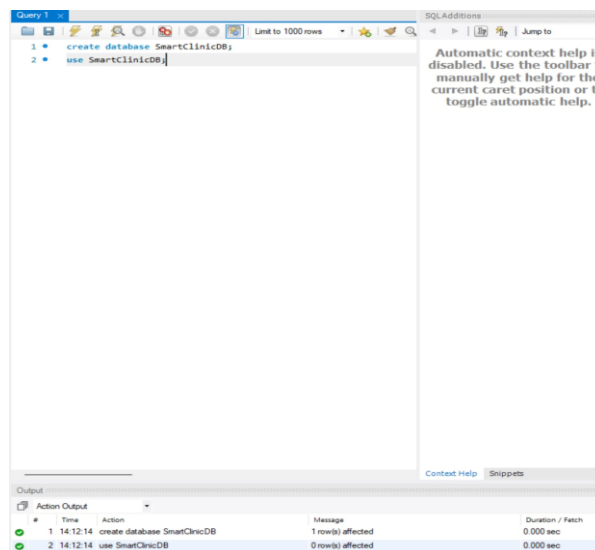


Figure 3: Creating the SmartClinicDB database.

– Table Creation

The database tables were created based on the ER and EER models developed in Task 1. Each table represents an entity or relationship within the Smart Clinic Database System. Appropriate data types, primary keys, foreign keys, and integrity constraints were defined to accurately implement the database design and maintain data consistency and referential integrity. The following sections present the implementation of each table, including the SQL code used for table creation, sample data insertion, and the corresponding output.

• PERSON Table

The **PERSON** table was implemented as the **superclass** in the EER model to store the shared information of both patients and doctors. It contains shared attributes, including the person's name and gender, which are inherited by the corresponding subclass tables.

SQL Code (Table Creation):

```
CREATE TABLE PERSON (  
    PersonID INT PRIMARY KEY,  
    Fname VARCHAR(50) NOT NULL,  
    Mname VARCHAR(50),  
    Lname VARCHAR(50) NOT NULL,  
    Gender CHAR(1) CHECK (Gender IN ('M', 'F'));
```



Figure 4: Creating the PERSON table.

Task 2: Database Implementation

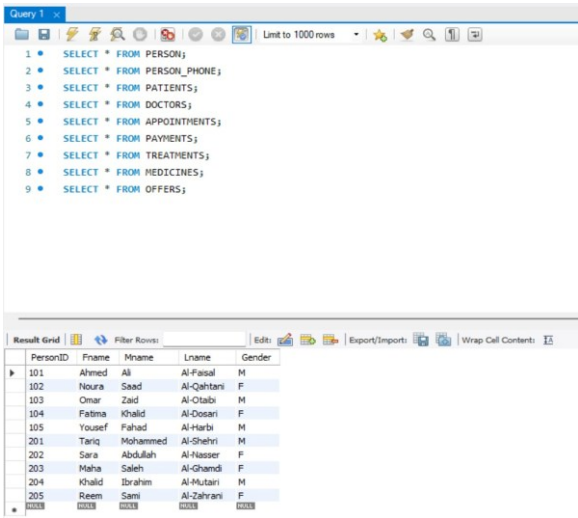
SQL Code (Data Insertion):

Sample records were inserted into the PERSON table to provide the common information for both patients and doctors before populating the subclass tables.

```
INSERT INTO PERSON (PersonID, Fname, Mname, Lname, Gender) VALUES
(101, 'Ahmed', 'Ali', 'Al-Faisal', 'M'),
(102, 'Noura', 'Saad', 'Al-Qahtani', 'F'),
(103, 'Omar', 'Zaid', 'Al-Otaibi', 'M'),
(104, 'Fatima', 'Khalid', 'Al-Dosari', 'F'),
(105, 'Yousef', 'Fahad', 'Al-Harbi', 'M'),
(201, 'Tariq', 'Mohammed', 'Al-Shehri', 'M'),
(202, 'Sara', 'Abdullah', 'Al-Nasser', 'F'),
(203, 'Maha', 'Saleh', 'Al-Ghamdi', 'F'),
(204, 'Khalid', 'Ibrahim', 'Al-Mutairi', 'M'),
(205, 'Reem', 'Sami', 'Al-Zahrani', 'F');
```



Figure 5: Inserting records into the PERSON table.



Output

The following output confirms that the records were successfully inserted into the PERSON table.

Table 1: Contents of the PERSON table.

Task 2: Database Implementation

• PERSON_PHONE Table

The **PERSON_PHONE** table was created to represent the multivalued phone attribute associated with the **PERSON** superclass. This implementation allows each person to have one or more phone numbers while maintaining database normalization and ensuring data consistency.

SQL Code (Table Creation):

```
CREATE TABLE PERSON_PHONE (
    PersonID INT,
    Phone VARCHAR(15),
    PRIMARY KEY (PersonID, Phone),
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE);
```

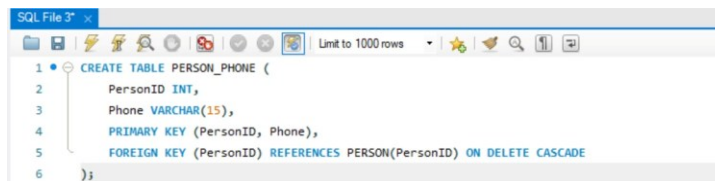


Figure 6: Creating the PERSON_PHONE table.

SQL Code (Data Insertion):

Sample phone numbers were inserted into the PERSON_PHONE table to associate contact numbers with the corresponding persons in the PERSON table

```
INSERT INTO PERSON_PHONE (PersonID, Phone) VALUES
(101, '0501111111'),
(102, '0502222222'),
(103, '0503333333'),
(201, '0506666666'),
(202, '0507777777');
```

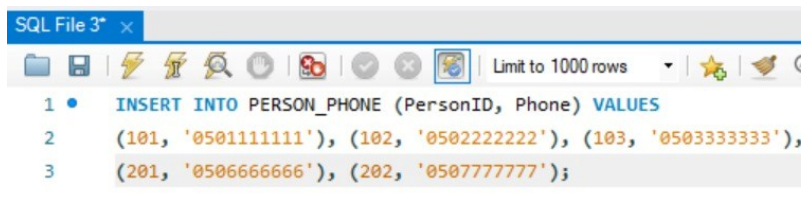
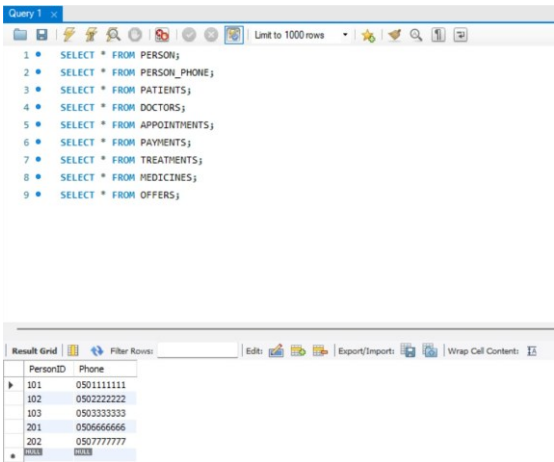


Figure 7: Inserting records into the PERSON_PHONE table.



Output

The following output confirms that the phone numbers were successfully stored in the PERSON_PHONE table.

Table 2: Contents of the PERSON_PHONE table.

• PATIENTS Table

The PATIENTS table was implemented as a subclass of the PERSON superclass. It stores patient-specific information, including the patient's insurance details, while maintaining a relationship with the PERSON table through the PersonID foreign key. This implementation reflects the specialization defined in the EER model and preserves the inheritance relationship between the superclass and subclass.

SQL Code (Table Creation):

```

CREATE TABLE PATIENTS (
    FileID INT PRIMARY KEY,
    PersonID INT UNIQUE,
    Insurance VARCHAR(100),
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE);

```

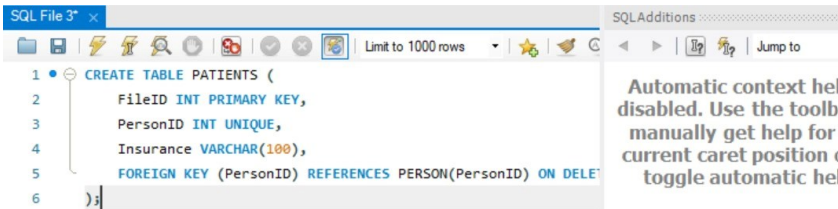


Figure 8: Creating the PATIENTS table.

Task 2: Database Implementation

SQL Code (Data Insertion):

Sample patient records were inserted into the PATIENTS table to associate each patient with an existing person and record the corresponding insurance information.

```
INSERT INTO PATIENTS (FileID, PersonID, Insurance) VALUES
(1, 101, 'Tawuniya'),
(2, 102, 'Bupa'),
(3, 103, 'Medgulf'),
(4, 104, 'Tawuniya'),
(5, 105, 'Malath');
```

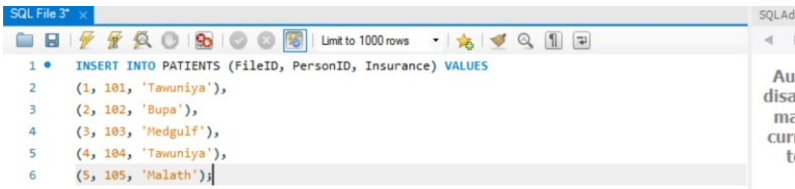
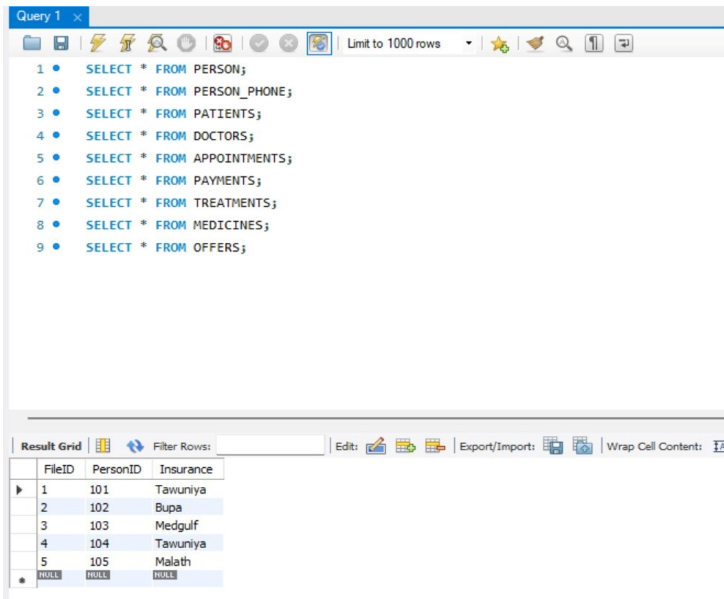


Figure 9: Inserting records into the PATIENTS table.



Output
The following output confirms that the patient records were successfully inserted into the PATIENTS table.

Table 3: Contents of the PATIENTS table.

Task 2: Database Implementation

• DOCTORS Table

The **DOCTORS** table was implemented as a **subclass** of the **PERSON** superclass. It stores doctor-specific information, including the doctor's specialization, while maintaining a relationship with the **PERSON** table through the **PersonID** foreign key. This implementation follows the specialization defined in the EER model and ensures that each doctor inherits the common attributes stored in the **PERSON** table.

SQL Code (Table Creation):

```
CREATE TABLE DOCTORS (
    DoctorID INT PRIMARY KEY,
    PersonID INT UNIQUE,
    Specialization VARCHAR(100) NOT NULL,
    FOREIGN KEY (PersonID) REFERENCES PERSON(PersonID) ON DELETE CASCADE);
```

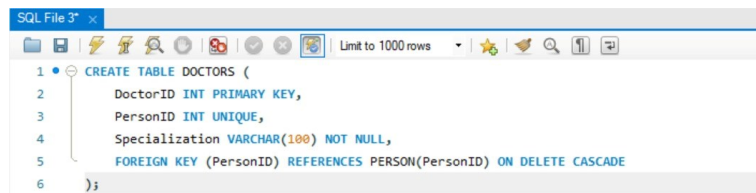


Figure 10: Creating the DOCTORS table.

SQL Code (Data Insertion):

Sample doctor records were inserted into the DOCTORS table to associate each doctor with an existing person and record the corresponding medical specialization.

```
INSERT INTO DOCTORS (DoctorID, PersonID, Specialization) VALUES
(10, 201, 'Cardiology'),
(20, 202, 'Dermatology'),
(30, 203, 'Pediatrics'),
(40, 204, 'General Practice'),
(50, 205, 'Orthopedics');
```

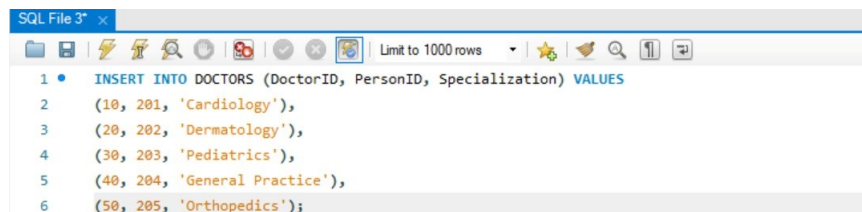
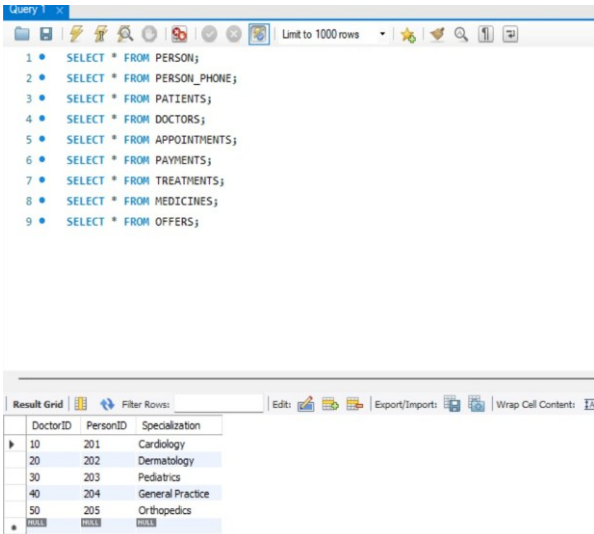


Figure 11: Inserting records into the DOCTORS table.



Output

The following output confirms that the doctor’s records were successfully inserted into the DOCTORS table.

Table 4: Contents of the DOCTORS table.

• APPOINTMENTS Table

The **APPOINTMENTS** table was created to store appointment information and represent the relationship between patients and doctors. Each appointment is associated with one patient and one doctor through foreign key references, ensuring accurate scheduling and maintaining referential integrity within the database.

SQL Code (Table Creation):

```
CREATE TABLE APPOINTMENTS (  
    ApptID INT PRIMARY KEY,  
    Appt_Date DATE NOT NULL,  
    Appt_Time TIME NOT NULL,  
    FileID INT NOT NULL,  
    DoctorID INT NOT NULL,  
    FOREIGN KEY (FileID) REFERENCES PATIENTS(FileID) ON DELETE CASCADE,  
    FOREIGN KEY (DoctorID) REFERENCES DOCTORS(DoctorID) ON DELETE CASCADE);
```

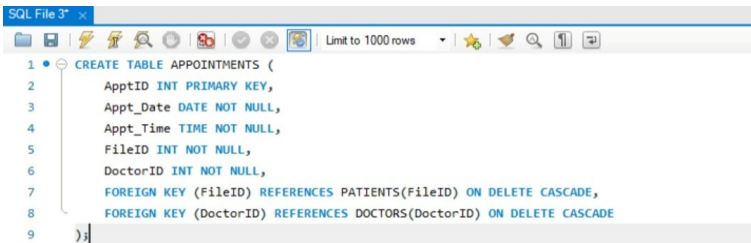


Figure 12: Creating the APPOINTMENTS table.

Task 2: Database Implementation

SQL Code (Data Insertion):

Sample appointment records were inserted into the APPOINTMENTS table to schedule appointments between patients and doctors on specific dates and times.

```
INSERT INTO APPOINTMENTS (ApptID, Appt_Date, Appt_Time, FileID, DoctorID) VALUES
(1001, '2026-08-01', '09:00:00', 1, 10),
(1002, '2026-08-01', '10:30:00', 2, 20),
(1003, '2026-08-02', '14:00:00', 3, 30),
(1004, '2026-08-03', '11:15:00', 4, 40),
(1005, '2026-08-04', '16:45:00', 5, 50);
```

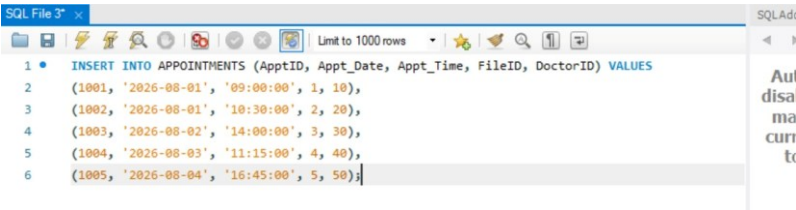
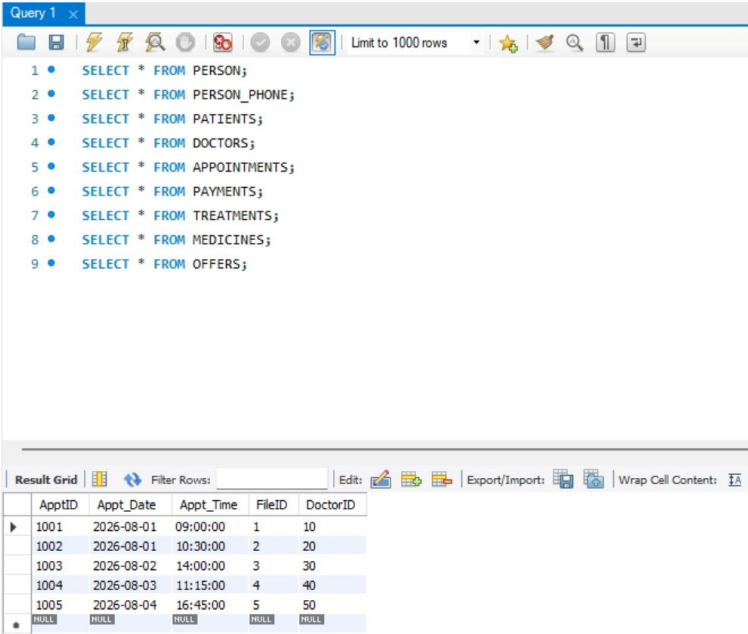


Figure 13: Inserting records into the APPOINTMENTS table.



Output
The following output confirms that the appointment records were successfully inserted into the APPOINTMENTS table.

Table 5: Contents of the APPOINTMENTS table.

Task 2: Database Implementation

• PAYMENTS Table

The **PAYMENTS** table was created to store payment information associated with patients. It records the payment amount, payment date, and the corresponding patient responsible for each transaction through a foreign key relationship. This implementation allows payment records to be managed efficiently while maintaining data consistency.

SQL Code (Table Creation):

```
CREATE TABLE PAYMENTS (
    PayID INT PRIMARY KEY,
    Total DECIMAL(10,2) NOT NULL,
    Pay_Date DATE NOT NULL,
    FileID INT NOT NULL,
    FOREIGN KEY (FileID) REFERENCES PATIENTS(FileID) ON DELETE CASCADE);
```

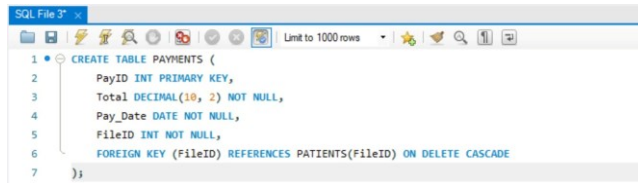


Figure 14: Creating the PAYMENTS table.

SQL Code (Data Insertion):

Sample payment records were inserted into the PAYMENTS table to record payments made by patients for the medical services they received.

```
INSERT INTO PAYMENTS (PayID, Total, Pay_Date, FileID) VALUES
(501, 150.00, '2026-08-01', 1),
(502, 300.50, '2026-08-01', 2),
(503, 100.00, '2026-08-02', 3),
(504, 50.00, '2026-08-03', 4),
(505, 450.75, '2026-08-04', 5);
```

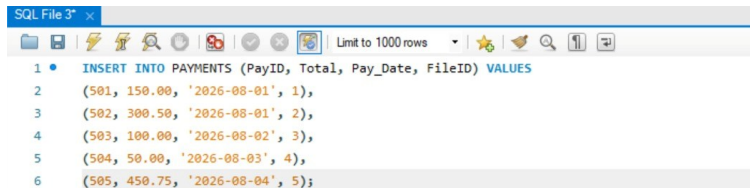
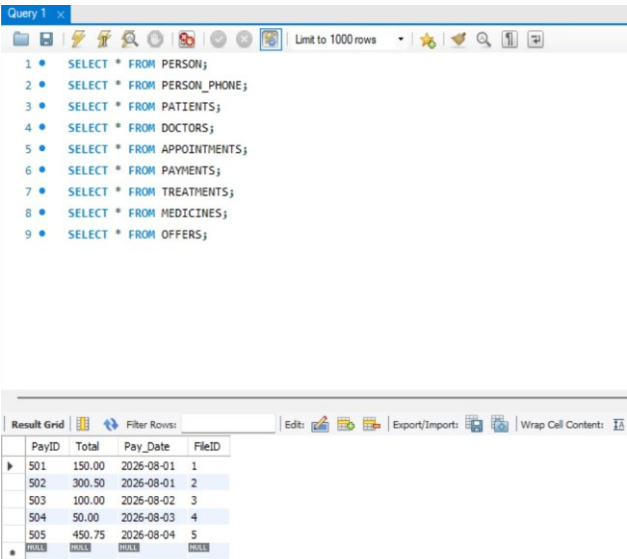


Figure 15: Inserting records into the PAYMENTS table.



Output

The following output confirms that the payment records were successfully inserted into the PAYMENTS table.

Table 6: Contents of the PAYMENTS table.

• TREATMENTS Table

The TREATMENTS table was created to store information about the medical treatments provided during patient appointments. Each treatment is linked to a specific appointment through a foreign key relationship, ensuring that every treatment is associated with the appropriate patient visit and accurately reflects the clinic's operations.

SQL Code (Table Creation):

```
CREATE TABLE TREATMENTS (  
    TreatID INT PRIMARY KEY,  
    Treat_Name VARCHAR(100) NOT NULL,  
    Start_date DATE NOT NULL,  
    End_date DATE,  
    ApptID INT NOT NULL,  
    FOREIGN KEY (ApptID) REFERENCES APPOINTMENTS(ApptID) ON DELETE  
    CASCADE);
```

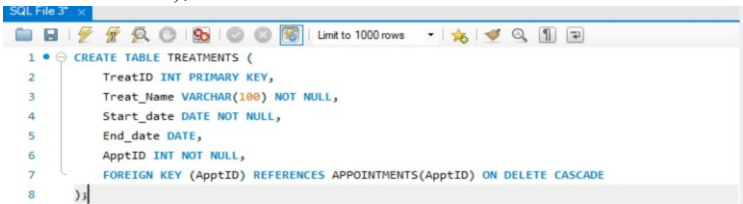


Figure 16: Creating the TREATMENTS table.

Task 2: Database Implementation

SQL Code (Data Insertion):

Sample treatment records were inserted into the TREATMENTS table to represent the medical services provided during scheduled appointments.

```
INSERT INTO TREATMENTS (TreatID, Treat_Name, Start_date, End_date, ApptID) VALUES
(701, 'ECG Monitoring', '2026-08-01', '2026-08-01', 1001),
(702, 'Laser Therapy', '2026-08-01', '2026-08-15', 1002),
(703, 'Vaccination', '2026-08-02', '2026-08-02', 1003),
(704, 'Routine Checkup', '2026-08-03', '2026-08-03', 1004),
(705, 'Physiotherapy', '2026-08-04', '2026-09-04', 1005);
```

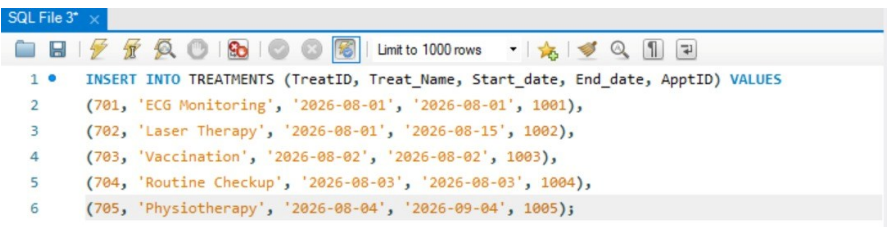
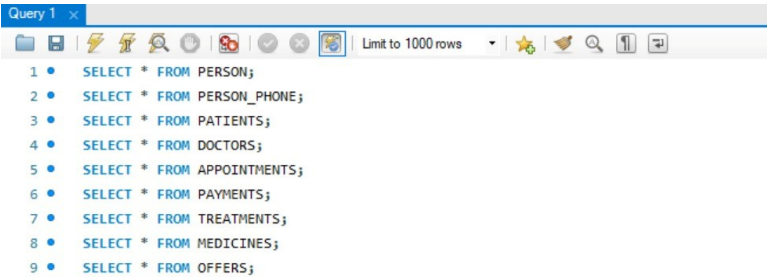


Figure 17:
Inserting records into the TREATMENTS table.



Output
The following output confirms that the treatment records were successfully inserted into the TREATMENTS table.

TreatID	Treat_Name	Start_date	End_date	ApptID
701	ECG Monitoring	2026-08-01	2026-08-01	1001
702	Laser Therapy	2026-08-01	2026-08-15	1002
703	Vaccination	2026-08-02	2026-08-02	1003
704	Routine Checkup	2026-08-03	2026-08-03	1004
705	Physiotherapy	2026-08-04	2026-09-04	1005
NULL	NULL	NULL	NULL	NULL

Table 7: Contents of the TREATMENTS table.

• MEDICINES Table

The **MEDICINES** table was created to store information about the medicines prescribed to patients during treatment. Each medicine is uniquely identified and includes its name and cost, allowing medicines to be associated with treatments through the OFFERS relationship while avoiding data duplication.

SQL Code (Table Creation):

```
CREATE TABLE MEDICINES (
    MedicineID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL UNIQUE,
    Cost DECIMAL(10,2) NOT NULL);
```

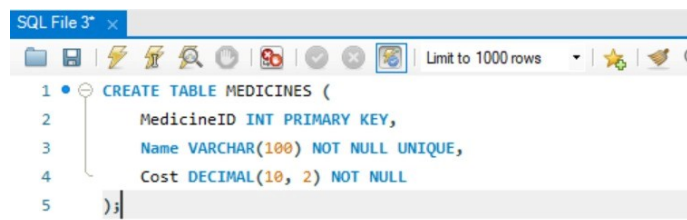


Figure 18: Creating the MEDICINES table.

SQL Code (Data Insertion):

Sample medicine records were inserted into the MEDICINES table to represent the medicines available for prescription within the clinic.

```
INSERT INTO MEDICINES (MedicineID, Name, Cost) VALUES
(901, 'Aspirin', 15.50),
(902, 'Isotretinoin', 120.00),
(903, 'Paracetamol', 10.00),
(904, 'Amoxicillin', 45.25),
(905, 'Ibuprofen', 20.00);
```

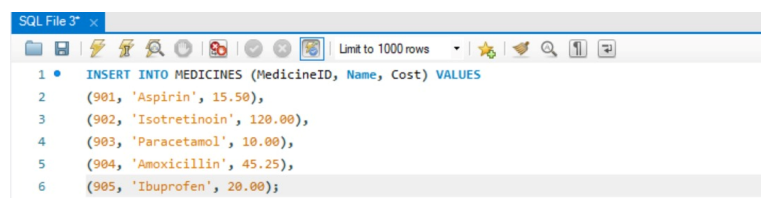
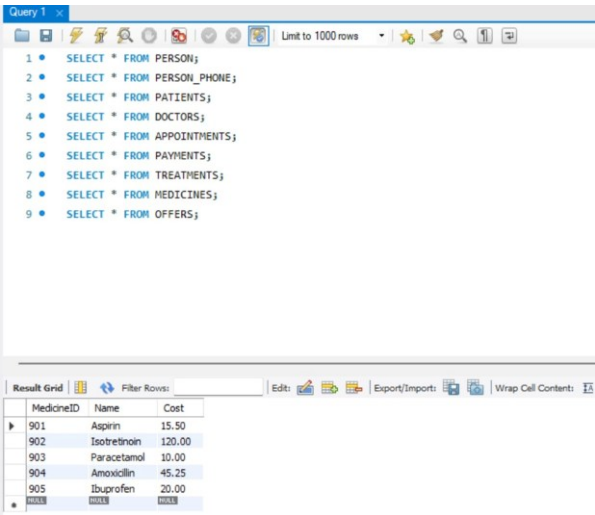


Figure 19: Inserting records into the MEDICINES table.



Output

The following output confirms that the medicine records were successfully inserted into the MEDICINES table.

Table 8: Contents of the MEDICINES table.

• OFFERS Table

The OFFERS table was implemented to represent the many-to-many (M:N) relationship between the TREATMENTS and MEDICINES tables. It stores the quantity of each medicine prescribed for a particular treatment, accurately implementing the relationship defined in the ER and EER models and ensuring that multiple medicines can be associated with a single treatment while each medicine can also be prescribed in multiple treatments.

SQL Code (Table Creation):

```
CREATE TABLE OFFERS (  
    TreatID INT,  
    MedicineID INT,  
    Quantity INT NOT NULL CHECK (Quantity > 0),  
    PRIMARY KEY (TreatID, MedicineID),  
    FOREIGN KEY (TreatID) REFERENCES TREATMENTS(TreatID) ON DELETE CASCADE,  
    FOREIGN KEY (MedicineID) REFERENCES MEDICINES(MedicineID) ON DELETE CASCADE);
```

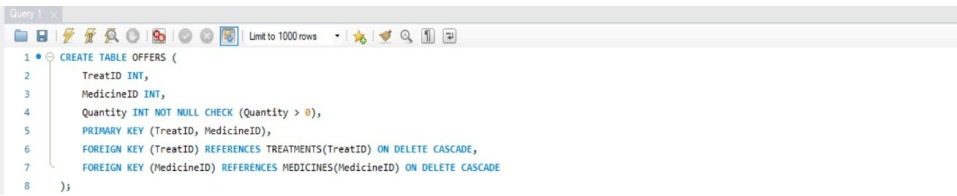


Figure 20: Creating the OFFERS table.

Task 2: Database Implementation

SQL Code (Data Insertion):

Sample records were inserted into the OFFERS table to associate treatments with the prescribed medicines and record the quantity required for each treatment.

```
INSERT INTO OFFERS (TreatID, MedicineID, Quantity) VALUES
(701, 901, 1),
(702, 902, 2),
(703, 903, 1),
(704, 904, 1),
(705, 905, 3);
```

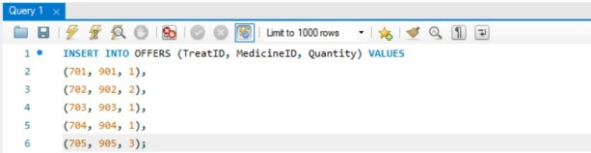
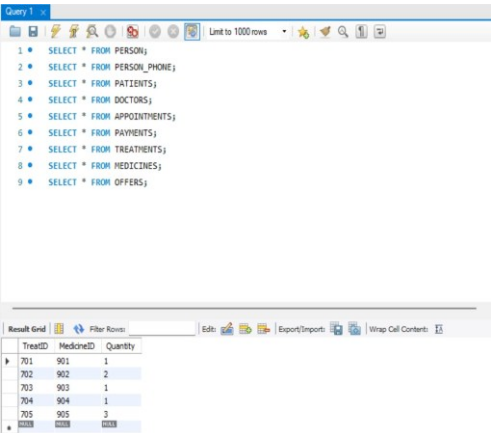


Figure 21: Inserting records into the OFFERS table.



Output

The following output confirms that the relationship records were successfully inserted into the OFFERS table.

Table 9: Contents of the OFFERS table.

– Summary

In this task, the Smart Clinic Database System was successfully implemented using MySQL based on the database design developed in Task 1. The implementation included creating the database, defining all required tables, establishing relationships through primary and foreign keys, applying appropriate constraints, and inserting sample records into each table. The completed implementation provides a functional and reliable database that is ready for performing SQL queries, creating views and triggers, and completing the remaining project tasks.

Task 3: SQL Operations

The SQL operations were performed using MySQL to demonstrate the functionality of the Smart Clinic Database System. Different SQL statements were implemented to retrieve and manipulate the stored data while verifying that the relationships established during the database implementation operate correctly. The following sections present the SQL code, corresponding outputs, and a brief description explaining the purpose of each query.

– SELECT Statements

SELECT statements were implemented to retrieve specific information from the database based on predefined conditions. These queries allow users to access only the required data without modifying the stored records.

- **Query 1**

This query retrieves the names and costs of medicines with a cost greater than 20 SAR. It is useful for identifying medicines with relatively higher prices.

SQL Code:

```
SELECT Name, Cost  
FROM MEDICINES  
WHERE Cost > 20;
```

- **Query 2**

This query retrieves the first name, middle name, last name, and gender of all female persons stored in the database.

SQL Code:

```
SELECT Fname, Mname, Lname, Gender  
FROM PERSON  
WHERE Gender = 'F';
```

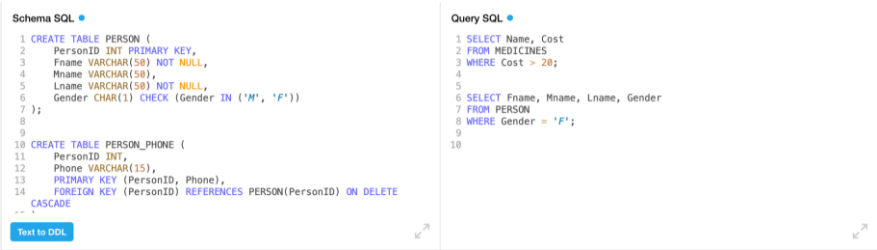


Figure 22: SQL code for the SELECT statements.

Output

Results

Query #1 Execution time: 0.25ms

Name	Cost
Isotretinoin	120.0
Amoxicillin	45.25

Query #2 Execution time: 0.22ms

Query 1 displays the medicines with a cost greater than 20 SAR, showing their names and costs.

Query #2 Execution time: 0.22ms

Fname	Mname	Lname	Gender
Noura	Saad	Al-Qahtani	F
Fatima	Khalid	Al-Dosari	F
Sara	Abdullah	Al-Nasser	F
Maha	Saleh	Al-Ghamdi	F
Reem	Sami	Al-Zahrani	F

Query2 displays the first name, middle name, last name, and gender of all female persons in the database.

Table 10: Results of the SELECT statements.

– JOIN Queries

JOIN queries were implemented to combine related data from multiple tables using their primary and foreign key relationships. These queries demonstrate how information stored across different tables can be retrieved as a single result.

- Query 1

This query joins the **PERSON**, **PATIENTS**, **APPOINTMENTS**, and **DOCTORS** tables to display each patient's name, appointment date and time, doctor's name, and specialization in a single result.

SQL Code:

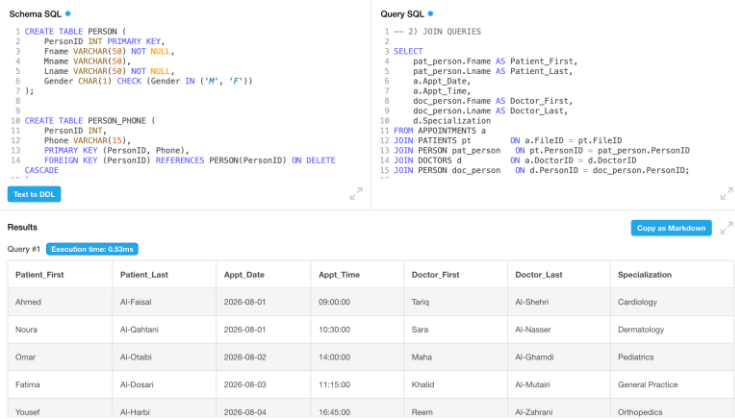
```
SELECT
    pat_person.Fname AS Patient_First,
    pat_person.Lname AS Patient_Last,
    a.Appt_Date,
    a.Appt_Time,
    doc_person.Fname AS Doctor_First,
    doc_person.Lname AS Doctor_Last,
    d.Specialization
FROM APPOINTMENTS a
JOIN PATIENTS pt ON a.FileID = pt.FileID
JOIN PERSON pat_person ON pt.PersonID = pat_person.PersonID
JOIN DOCTORS d ON a.DoctorID = d.DoctorID
JOIN PERSON doc_person ON d.PersonID = doc_person.PersonID;
```

Figure 23: SQL code for Query 1

Output

The following output displays each patient's appointment details together with the corresponding doctor's information.

Table11: Results of Query 1.



Patient_First	Patient_Last	Appt_Date	Appt_Time	Doctor_First	Doctor_Last	Specialization
Ahmed	Al-Faisal	2026-08-01	09:00:00	Tariq	Al-Shehri	Cardiology
Noura	Al-Qahtani	2026-08-01	10:30:00	Sara	Al-Nasser	Dermatology
Omar	Al-Otaibi	2026-08-02	14:00:00	Maha	Al-Ghamdi	Pediatrics
Fatima	Al-Dosari	2026-08-03	11:15:00	Khalid	Al-Mutairi	General Practice
Yousef	Al-Harbi	2026-08-04	16:45:00	Reem	Al-Zahrani	Orthopedics

• Query 2

This query joins the **TREATMENTS**, **OFFERS**, and **MEDICINES** tables to display the medicines prescribed for each treatment together with the prescribed quantity.

SQL Code:

```
SELECT
    t.Treat_Name,
    m.Name AS Medicine_Name,
    o.Quantity
FROM TREATMENTS t
JOIN OFFERS o ON t.TreatID = o.TreatID
JOIN MEDICINES m ON o.MedicineID = m.MedicineID;
```

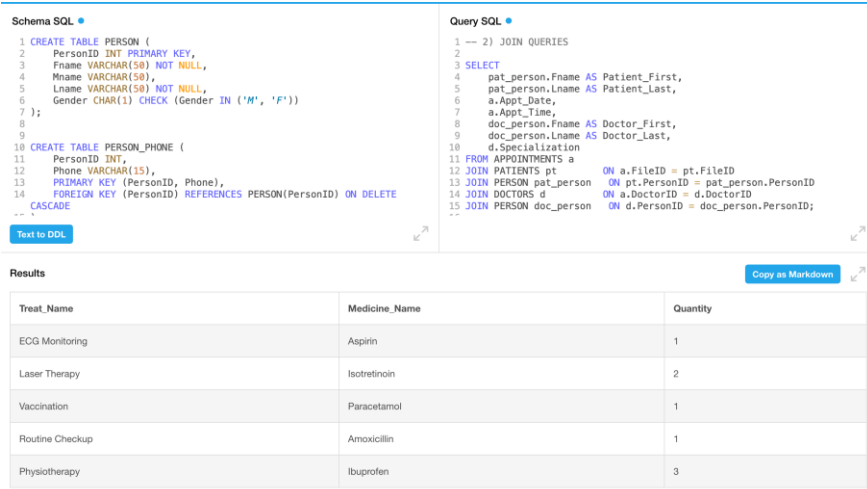


Figure 24: SQL code for Query 2.

Output

The following output displays the medicines associated with each treatment and the quantity prescribed.

Table 12: Results of Query 2.

– Nested Queries

Nested queries were implemented to retrieve information by using the result of one query as the input for another. These queries demonstrate how subqueries can be used to perform more advanced data retrieval operations.

- Query 1

This query uses nested subqueries to retrieve the names of patients who made payments greater than **100 SAR**.

SQL Code:

```
SELECT Fname, Lname
FROM PERSON
WHERE PersonID IN (
    SELECT PersonID
    FROM PATIENTS
    WHERE FileID IN (
        SELECT FileID
        FROM PAYMENTS
        WHERE Total > 100
    )
);
```



Figure 25: SQL code for Query 1.

Output

The following output displays the names of patients whose payments exceeded **100 SAR**.

Table 13: Results of Query 1.

- Query 2

This query uses nested subqueries to identify the treatment associated with the medicine **Aspirin** by first retrieving the medicine's ID and then locating the corresponding treatment.

SQL Code:

```
SELECT Treat_Name
FROM TREATMENTS
WHERE TreatID IN (
    SELECT TreatID
    FROM OFFERS
    WHERE MedicineID = (
        SELECT MedicineID
        FROM MEDICINES
        WHERE Name = 'Aspirin'
    )
);
```



Figure 26: SQL code for Query 2.

Output

The following output displays the treatment in which Aspirin was prescribed.

Table 14: Results of Query 2.

– Aggregate Functions with GROUP BY

Aggregate functions were implemented together with the **GROUP BY** clause to summarize data stored in the database. These queries demonstrate how SQL can be used to calculate statistical information for different groups of records.

- Query 1

This query counts the total number of appointments handled by each doctor using the **COUNT()** aggregate function together with the **GROUP BY** clause.

SQL Code:

```
SELECT DoctorID, COUNT(*) AS Num_Appointments
FROM APPOINTMENTS
GROUP BY DoctorID;
```

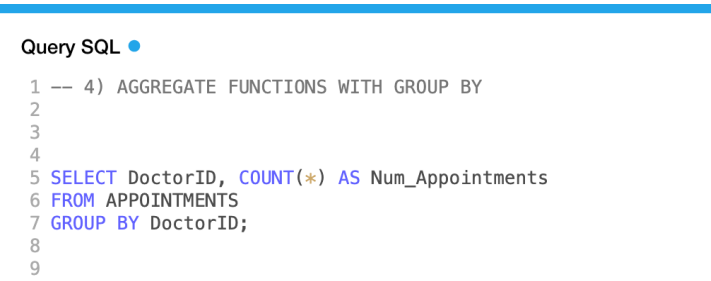


Figure 27: SQL code for Query 1.

Results

Query #1 Execution time: 0.31ms

Copy as Markdown

DoctorID	Num_Appointments
10	1
20	1
30	1
40	1
50	1

Table 15: Results of Query 1.

Output

The following output displays the total number of appointments assigned to each doctor.

- Query 2

This query calculates the total and average payments made by each patient using the **SUM()** and **AVG()** aggregate functions together with the **GROUP BY** clause.

SQL Code:

```
SELECT FileID,
        SUM(Total) AS Total_Paid,
        AVG(Total) AS Avg_Payment
FROM PAYMENTS
GROUP BY FileID;
```

```
8
9
10 SELECT FileID, SUM(Total) AS Total_Paid, AVG(Total) AS
    Avg_Payment
11 FROM PAYMENTS
12 GROUP BY FileID;
13
14
```

Figure 28: SQL code for Query 2.

Results			Copy as Markdown
Query #2			Execution time: 0.18ms
FileID	Total_Paid	Avg_Payment	
1	150.0	150.0	
2	300.5	300.5	
3	100.0	100.0	
4	50.0	50.0	
5	450.75	450.75	

Table 16: Results of Query 2.

Output

The following output displays the total and average payment amount for each patient.

– UPDATE and DELETE Statements

The following SQL statements demonstrate how existing records can be modified and removed from the database. The **UPDATE** statement changes existing data, while the **DELETE** statement removes unnecessary records from a table.

- **UPDATE Statement**

This statement updates the cost of **Aspirin** by increasing its price by **10%**, demonstrating how existing records can be modified within the database.

SQL Code:

```
UPDATE MEDICINES
SET Cost = Cost * 1.10
WHERE MedicineID = 901;

SELECT *FROM MEDICINES WHERE MedicineID = 901;
```

Query SQL ●

```
1 -- 5) UPDATE AND DELETE STATEMENTS
2
3 UPDATE MEDICINES
4 SET Cost = Cost * 1.10
5 WHERE MedicineID = 901;
6
7
8 SELECT * FROM MEDICINES WHERE MedicineID = 901;
9
```

Figure 29: SQL code for the UPDATE statement.

Query #2 Execution time: 0.19ms

MedicineID	Name	Cost
901	Aspirin	17.05

Table 17: Results of the UPDATE statement.

Output

The following output confirms that the cost of **Aspirin** was successfully updated.

• DELETE Statement

This statement removes a phone number associated with a specific person from the **PERSON_PHONE** table, demonstrating how records can be deleted while maintaining database integrity.

SQL Code:

```
DELETE FROM PERSON_PHONE
WHERE PersonID = 103
AND Phone = '0503333333';
SELECT * FROM PERSON_PHONE WHERE PersonID = 103;
```

```
10
11 DELETE FROM PERSON_PHONE
12 WHERE PersonID = 103 AND Phone = '0503333333';
13
14
15 SELECT * FROM PERSON_PHONE WHERE PersonID = 103;
```

Figure 30: SQL code for the DELETE statement.

Output

The following output confirms that the specified phone number was successfully removed from the **PERSON_PHONE** table.

Query #4 Execution time: 0.2ms

PersonID	Phone
There are no results to be displayed.	

Table 18: Results of the DELETE statement.

– VIEW

A view was created to simplify access to frequently used information by combining data from multiple related tables into a single virtual table. This allows users to retrieve the required information without rewriting complex JOIN queries.

The following view, **PatientAppointmentView**, combines information from the **PERSON**, **PATIENTS**, **APPOINTMENTS**, and **DOCTORS** tables to display patient names together with their appointment details and the doctor's specialization.

SQL Code:

```
CREATE VIEW PatientAppointmentView AS
SELECT
    p.Fname AS Patient_First,
    p.Lname AS Patient_Last,
    a.Appt_Date,
    a.Appt_Time,
    d.Specialization
FROM APPOINTMENTS a
JOIN PATIENTS pt ON a.FileID = pt.FileID
JOIN PERSON p ON pt.PersonID = p.PersonID
JOIN DOCTORS d ON a.DoctorID = d.DoctorID;
SELECT * FROM PatientAppointmentView;
```

Query SQL ●

```
1 -- 6) VIEW
2
3 CREATE VIEW PatientAppointmentView AS
4 SELECT
5     p.Fname AS Patient_First,
6     p.Lname AS Patient_Last,
7     a.Appt_Date,
8     a.Appt_Time,
9     d.Specialization
10 FROM APPOINTMENTS a
11 JOIN PATIENTS pt ON a.FileID = pt.FileID
12 JOIN PERSON p ON pt.PersonID = p.PersonID
13 JOIN DOCTORS d ON a.DoctorID = d.DoctorID;
14 SELECT * FROM PatientAppointmentView;
15
```

Figure 31: SQL code for creating the PatientAppointmentView.

Output

The following output displays the records returned by the **PatientAppointmentView**, showing each patient's appointment details together with the doctor's specialization.

Text to DDL

Results

Copy as Markdown

Patient_First	Patient_Last	Appt_Date	Appt_Time	Specialization
Ahmed	Al-Faisal	2026-08-01	09:00:00	Cardiology
Noura	Al-Qahtani	2026-08-01	10:30:00	Dermatology
Omar	Al-Otaibi	2026-08-02	14:00:00	Pediatrics
Fatima	Al-Dosari	2026-08-03	11:15:00	General Practice
Yousef	Al-Harbi	2026-08-04	16:45:00	Orthopedics

Table 19: Results of the PatientAppointmentView.

– TRIGGER

A trigger was implemented to automatically record newly scheduled appointments in a log table. This provides a simple audit trail by recording appointment activity without requiring manual data entry.

The trigger **trg_after_appointment_insert** automatically inserts a new record into the **APPOINTMENT_LOG** table whenever a new appointment is added to the **APPOINTMENTS** table.

SQL Code:

```
CREATE TABLE APPOINTMENT_LOG (  
    LogID INT AUTO_INCREMENT PRIMARY KEY,  
    ApptID INT,  
    Action_Type VARCHAR(50),  
    Action_Date DATETIME DEFAULT CURRENT_TIMESTAMP);  
  
CREATE TRIGGER trg_after_appointment_insert  
AFTER INSERT ON APPOINTMENTS  
FOR EACH ROW  
BEGIN  
    INSERT INTO APPOINTMENT_LOG (ApptID, Action_Type)  
    VALUES (NEW.ApptID, 'New appointment scheduled');  
END;  
  
INSERT INTO APPOINTMENTS  
(ApptID, Appt_Date, Appt_Time, FileID, DoctorID)  
VALUES  
(1006, '2026-08-05', '13:00:00', 1, 20);  
  
SELECT * FROM APPOINTMENT_LOG;
```

Query SQL

```
1 -- 7) TRIGGER
2 CREATE TABLE APPOINTMENT_LOG (
3     LogID INT AUTO_INCREMENT PRIMARY KEY,
4     ApptID INT,
5     Action_Type VARCHAR(50),
6     Action_Date DATETIME DEFAULT CURRENT_TIMESTAMP
7 );
8 CREATE TRIGGER trg_after_appointment_insert
9 AFTER INSERT ON APPOINTMENTS
10 FOR EACH ROW
11 BEGIN
12     INSERT INTO APPOINTMENT_LOG (ApptID, Action_Type)
13     VALUES (NEW.ApptID, 'New appointment scheduled');
14 END;
15 INSERT INTO APPOINTMENTS (ApptID, Appt_Date, Appt_Time,
16     FileID, DoctorID)
17     VALUES (1006, '2026-08-05', '13:00:00', 1, 20);
18 SELECT * FROM APPOINTMENT_LOG;
```

Figure 32: SQL code for creating the trigger and testing its execution.

Output

The following output confirms that a new record was automatically inserted into the **APPOINTMENT_LOG** table after adding a new appointment, verifying that the trigger executed successfully.

Query #4 Execution time: 0.28ms

LogID	ApptID	Action_Type	Action_Date
1	1006	New appointment scheduled	2026-08-04 18:22:41

Table 20: Results of the trigger execution.

Summary

The SQL operations successfully demonstrated the functionality of the Smart Clinic Database System. Various SQL statements were implemented to retrieve, summarize, update, and delete data while maintaining the relationships established during database implementation. In addition, a **VIEW** was created to simplify data retrieval, and a **TRIGGER** was implemented to automatically log newly scheduled appointments. These operations verify that the database supports efficient data management and fulfills the functional requirements of the Smart Clinic Database System.

Task 4: Reflection

Challenge Encountered

One of the main challenges our group faced during the project was implementing the relationships between the database tables while maintaining referential integrity. Since our Smart Clinic Database System contained several related entities, including patients, doctors, appointments, treatments, medicines, and payments, we had to make sure the tables were created in the correct order and that the relationships were defined properly. During the implementation, we encountered some foreign key errors because certain tables referenced other tables that had not been created yet. We identified the problem by reviewing the SQL error messages and comparing our code with the ER and EER diagrams from Task 1. We then reorganized the order in which the tables were created, making sure the parent tables were created before the child tables. After making these changes, we were able to implement the database successfully and verify that the relationships were working correctly.

Chosen Approach

We chose to implement the PERSON superclass with the PATIENTS and DOCTORS subclasses because this approach accurately represented the EER model created during the database design phase. We also considered storing all person information in a single table, but this would have introduced unnecessary data redundancy and several unused attributes. Although the superclass and subclass approach required us to use additional JOIN operations when retrieving information, we found that it kept the database more organized and reduced unnecessary repetition.

Use of AI Tools

During this project, we used ChatGPT as a supporting tool for a few specific tasks, such as understanding some SQL concepts, getting guidance on using GitHub, and improving the wording of parts of the report. We also used it to clarify questions whenever we were unsure about a step in the implementation. The database design, SQL implementation, testing, and final verification were completed by our group using MySQL Workbench, with ChatGPT used for guidance where needed.

Future Improvements

If we were to complete this project again with more time and resources, we would expand the Smart Clinic Database System by adding features such as patient medical history, prescription management, staff records, and billing reports. We would also create more advanced SQL queries, additional views and triggers, and develop a simple graphical user interface to make the system more practical and closer to a real-world clinic management application. These improvements would enhance both the functionality and usability of the system while providing a more comprehensive database solution.

Final Project Overview

Smart Clinic Database System

- **Group Members**

- Sara Khalid Alnwaihel
- Lena Turki Alquraini
- Jana Mousa Haqawi
- Mayar Abdulrahman Alshehri

- **Project Completion**

During this project, our group successfully designed and implemented the Smart Clinic Database System using MySQL. We completed the database design by developing the ER and EER diagrams, identifying the required entities, relationships, and constraints, and implementing the database schema. The database was then populated with sample records, and various SQL operations were performed, including SELECT statements, JOIN queries, nested queries, aggregate functions, UPDATE and DELETE statements, as well as the creation of a VIEW and a TRIGGER. Finally, the project report and reflection were completed, resulting in a fully functional database system that satisfied the project requirements.

- **Key Achievement**

One of the key achievements of this project was successfully translating the conceptual ER and EER designs into a working relational database while maintaining data integrity through the use of primary keys, foreign keys, and appropriate constraints. This ensured that the implemented database accurately reflected the Smart Clinic Database System requirements.

- **Group Member Contributions**

<i>Group Member</i>	<i>Contribution</i>
<i>Sara Alnwaihel</i>	Completed Task 1 by designing the ER and EER diagrams, defining entities, relationships, and design assumptions.
<i>Lena Alquraini</i>	Completed Task 2 by implementing the database in MySQL, creating the tables, constraints, and inserting sample data.
<i>Jana Haqawi</i>	Completed Task 3 by implementing SQL operations, including SELECT, JOIN, nested queries, aggregate functions, UPDATE, DELETE, VIEW, and TRIGGER.
<i>Mayar Alshehri</i>	Completed Task 4 by writing the reflection, organizing the report, reviewing the project documentation, and coordinating the final submission.

Project Artifacts

Live Report Document: [[Google Doc Smart Clinic Database System Report](#)]

GitHub/GitLab Repository: [[DS350 Smart Clinic Database System GitHub Repository](#)]